Search Typeahead System

HLD101 Assignment — Project Report

A prefix typeahead system with an in-memory top-K trie, a distributed Redis cache sharded by consistent hashing, recency-aware trending, and batched search-count writes, served through a FastAPI backend and a React UI.

Repository	github.com/G-SaiVishwas/search-typeahead-hld
Stack	FastAPI + SQLite + Redis x3 + React (Vite)
Dataset	AOL User Session Collection (500k), Kaggle
Scale	1,244,453 seed queries / 2,969,752 raw events

1. Architecture

The system is read-optimized and write-buffered. Reads are served from a Redis cache, then an in-memory trie, then SQLite. Writes are accepted instantly and applied to SQLite in aggregated batches.

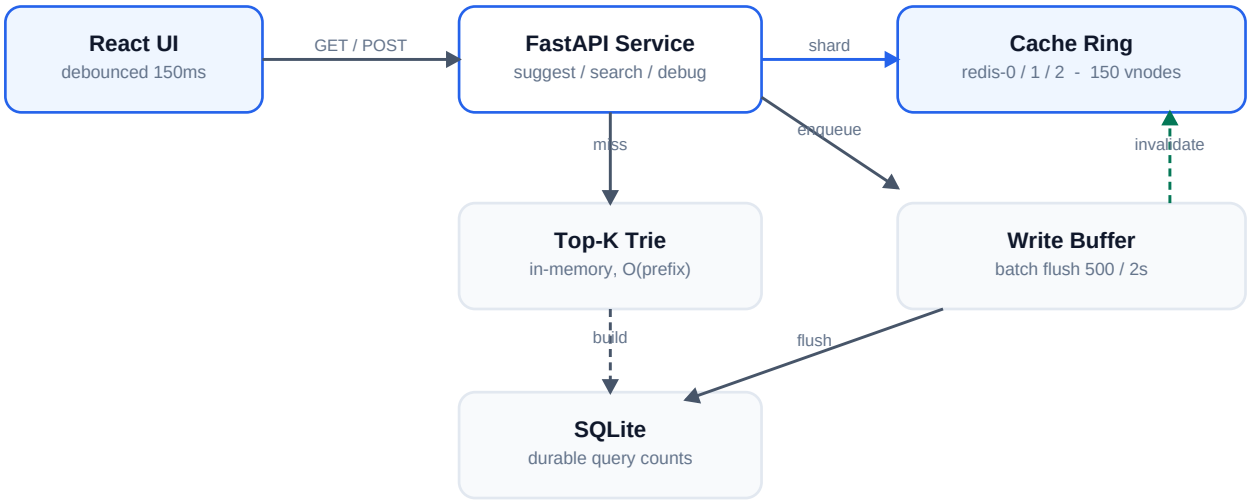

Figure 1 — Request flow across UI, API, cache ring, trie, buffer, and store.

Read path (GET /suggest)

- Normalize the prefix (trim, lowercase, collapse whitespace).
- Hash the key `suggest:{mode}:{prefix}` onto the ring and route to the owning Redis node.
- Cache hit → return cached top-K JSON immediately.
- Cache miss → walk the trie for the precomputed top-K (O(prefix length), no sort), populate the cache with a TTL, and return. Long-tail prefixes fall back to an indexed SQLite prefix scan.

Write path (POST /search)

- The event is appended to an in-memory buffer and the API returns `{"message":"Searched"}` without blocking.

- A background thread flushes every 500 events or 2 seconds, aggregating duplicates into per-query deltas.
- One SQLite transaction applies all deltas; trie counts are updated in place and affected prefix keys are invalidated in Redis.

2. Dataset Source & Loading

Source: AOL User Session Collection (500k) — real anonymized web search logs from March–May 2006, published on Kaggle: kaggle.com/datasets/dineshydv/aol-user-session-collection-500k.

Two derived CSV files are used:

File	Rows	Columns	Role
typeahed_dataset.csv	1,244,453	Query, Global/Weekly/Daily Count, Trending Score	Seed counts
raw_queries.csv	2,969,752	Query, QueryTime	Replay events / recency

The verified trending formula in the seed file is `0.6·Global + 0.3·Weekly + 0.1·Daily`.

Download links (derived CSV files)

File	Google Drive link
raw_queries.csv	https://drive.google.com/file/d/1XIbyjLBMxoSptTvZXeK65uIVr-RJ2LWZ/view?usp=sharing
typeahed_dataset.csv	https://drive.google.com/file/d/1931-OYamJ8ggTzkpPG1nt1R_SDbh1HDg/view?usp=sharing

Kaggle original: kaggle.com/datasets/dineshydv/aol-user-session-collection-500k

Loading instructions

Place both CSVs in `data/`, then start the stack. Ingestion runs automatically on first launch; the trie is built from SQLite at backend startup.

```
git clone & cd search-typeahead-hld
git lfs install # raw_queries.csv is tracked via Git LFS (>100MB)
./start.sh # Redis (Docker or local) + ingest + backend + frontend
# or manually:
python backend/scripts/ingest.py # bulk-load typeahed_dataset.csv → SQLite
```

Ingestion bulk-inserts in 50k-row transactions (~1.24M rows in ~36s) and creates `data/typeahead.db`.

3. API Documentation

Endpoint	Purpose
GET /suggest?q=&limit=&mode=	Top-K prefix suggestions (basic|trending)
POST /search	Submit a search; buffered count update
GET /cache/debug?prefix=	Node ownership, ring hash/position, hit/miss
GET /trending?limit=&mode=	Global trending (basic|trending)
GET /trending/compare?prefix=	Side-by-side basic vs trending ranking
POST /cache/demo/rebalance	Consistent-hashing remap demonstration
POST /batch/flush	Force-flush the write buffer (demo/test)
GET /metrics	Latency p50/p95/p99, hit rate, write reduction

GET /health	Trie size, DB rows, per-node Redis status

Example: GET /suggest?q=goog&limit=3

```
{
  "prefix": "goog", "mode": "basic",
  "cache": {"node": "redis-2", "hit": true},
  "suggestions": [
    {"query": "google", "count": 32948, "score": 20646.5},
    {"query": "google.com", "count": 8323, "score": 5268.7}
  ]
}
```

Example: GET /cache/debug?prefix=goog

```
{ "enabled": true, "assigned_node": "redis-2", "hit": true,
  "ring_position": 57, "total_vnodes": 450,
  "nodes": ["redis-0", "redis-1", "redis-2"] }
```

Input edge cases: empty/missing q returns an empty list with HTTP 200; matching is case-insensitive; no-match prefixes return an empty list; empty `POST /search` is rejected with 422.

4. Design Choices & Trade-offs

Decision	Why	Trade-off
Trie with precomputed top-K	O(prefix) lookup, no query-time sort	~ a few hundred MB; rebuilt at startup
Load global_count >= 10 into trie	Fast startup (~42k hot queries)	Long tail served via SQLite fallback
SQLite primary store	Zero-setup, single file, durable	Not horizontally scalable
3 standalone Redis + own ring	Consistent hashing is explicit/debuggable	Manual ring vs Redis Cluster
Batch writes (500 / 2s)	Collapse bursts; ~1000:1 read:write	Crash before flush loses buffer
Dual ranking modes	Basic (count) + recency-aware score	Two code paths to maintain

Failure mode & mitigations

Buffered events live in memory until flush, so a crash before flush loses pending counts. Mitigations: an append-only WAL, a durable queue (Redis Streams / Kafka), or a shorter flush interval — trading durability against write throughput. The cache degrades gracefully: if Redis is down, the API still serves from the trie and SQLite.

5. Performance Report

Measured locally (Apple Silicon) with all three Redis nodes running. Workload: 27 representative prefixes requested cold once then 20× warm (567 suggest requests), followed by a 5,000-event search replay.

Metric	Result
/suggest server-side p50	0.321 ms
/suggest server-side p95	0.644 ms
/suggest server-side p99	0.869 ms
Cache hit rate (warm)	95.8% (569 hits / 25 misses)
Search events replayed	5,000

SQLite write transactions	5
Write reduction ratio	0.999 (99.9% fewer writes)
Automated smoke tests	21 / 21 pass

Consistent hashing distribution

27 prefix keys mapped across the 3-node ring: redis-0 = 9, redis-1 = 10, redis-2 = 8. Adding a 4th node via `/cache/demo/rebalance` remapped 25% of sample keys — matching the ~1/N expectation for consistent hashing (N = 4).

Interpretation

- Server-side p99 stays under 1 ms because the trie returns precomputed top-K with no sorting at query time.
- After warmup ~96% of suggest requests are served from Redis without touching the trie or DB.
- 5,000 search events collapsed into 5 DB transactions, confirming the batch-write design.

Reproduce: `./start.sh` then `python backend/scripts/smoke_test.py` and `python backend/scripts/replay.py --limit 5000`; read `GET /metrics`.